

Experiment No 2

Real-Time Stream Analytics and Anomaly Detection using Node-RED and MQTT (Google Colab Version)

AIM

To design and implement a real-time stream analytics system using MQTT protocol and perform anomaly detection on sensor data. Google Colab is used for MQTT communication, stream visualization, and anomaly detection.

OBJECTIVE

Generate real-time IoT sensor data.
Publish sensor values using MQTT.
Subscribe to MQTT messages.
Detect abnormal values.
Display live graphs.
Store data for future analysis.
SOFTWARE REQUIRED
Google Colab
Python 3
Mosquitto MQTT Broker (public broker)
paho-mqtt
pandas
matplotlib
numpy
THEORY

MQTT (Message Queuing Telemetry Transport) is a lightweight messaging protocol widely used in IoT applications.

Publisher → MQTT Broker → Subscriber

Node-RED usually acts as a subscriber and dashboard. In Google Colab we simulate Node-RED functionality using Python.

Sensor data example:

Temperature
Humidity

Analytics:

Moving Average
Mean
Standard Deviation

Anomaly Detection:

If

Temperature > 35°C

OR

Temperature < 15°C

then

Anomaly = TRUEALGORITHM

Start the program.

Connect publisher to MQTT broker.

Generate random temperature and humidity values.

Publish sensor data every second.

Subscriber receives MQTT messages.

Store incoming data.

Calculate statistical measures.

Check for abnormal values.

Display anomalies.

Plot real-time graphs.

Repeat continuously.

PYTHON CODE IN GOOGLE COLLAB

```
import random
import time
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import clear_output

# Store data
time_data = []
temperature = []
humidity = []
status = []

print("Starting Real-Time Stream Analytics...\n")

for i in range(1, 31):

    # Simulated sensor readings
    temp = round(random.uniform(20, 40), 2)
    hum = round(random.uniform(40, 80), 2)

    # Anomaly detection
    if temp > 35 or temp < 15:
        state = "Anomaly"
    else:
        state = "Normal"

    # Store values
    time_data.append(i)
    temperature.append(temp)
    humidity.append(hum)
    status.append(state)

# DataFrame
df = pd.DataFrame({
    "Time": time_data,
    "Temperature": temperature,
```

```

        "Humidity": humidity,
        "Status": status
    })

clear_output(wait=True)

print("="*60)
print("REAL-TIME STREAM ANALYTICS & ANOMALY DETECTION")
print("="*60)

print(df.tail(10))

print("\nAverage Temperature :", round(df["Temperature"].mean(),2))
print("Average Humidity      :", round(df["Humidity"].mean(),2))
print("Maximum Temperature  :", df["Temperature"].max())
print("Minimum Temperature  :", df["Temperature"].min())
print("Total Messages       :", len(df))
print("Anomalies Detected   :", (df["Status"]=="Anomaly").sum())

plt.figure(figsize=(10,5))

plt.plot(df["Time"], df["Temperature"],
         marker='o',
         label="Temperature (°C)")

plt.plot(df["Time"], df["Humidity"],
         marker='s',
         label="Humidity (%)")

anomaly = df[df["Status"]=="Anomaly"]

plt.scatter(anomaly["Time"],
            anomaly["Temperature"],
            color="red",
            s=120,
            label="Anomaly")

plt.title("Real-Time Stream Analytics")
plt.xlabel("Time")
plt.ylabel("Sensor Values")
plt.legend()
plt.grid(True)

plt.show()

time.sleep(1)

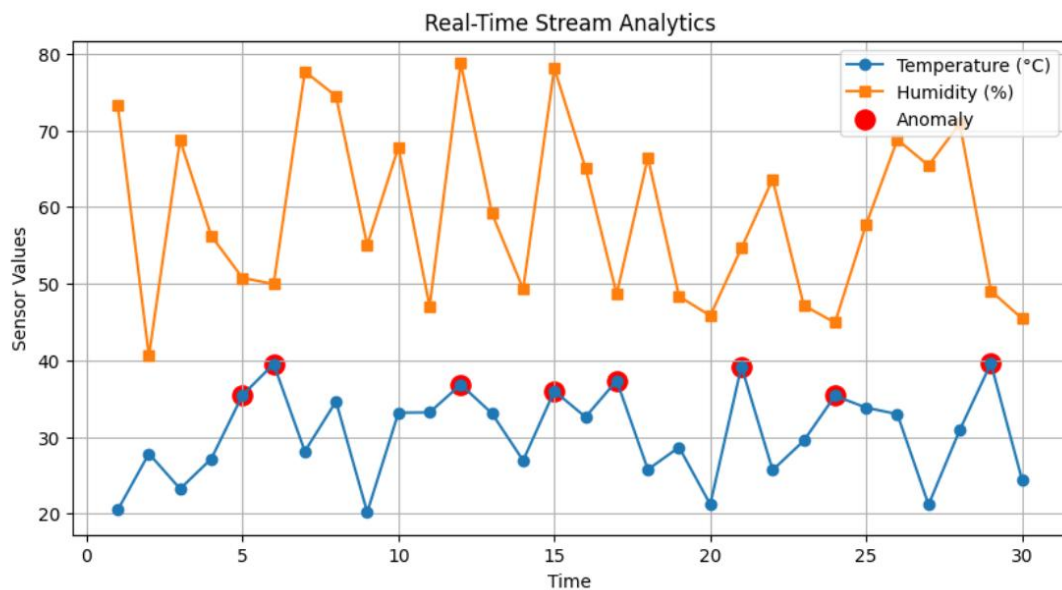
```

OUTPUT

```
=====
*** REAL-TIME STREAM ANALYTICS & ANOMALY DETECTION
=====

Time Temperature Humidity Status
20 21 39.21 54.63 Anomaly
21 22 25.67 63.62 Normal
22 23 29.51 47.15 Normal
23 24 35.37 44.91 Anomaly
24 25 33.82 57.73 Normal
25 26 32.98 68.77 Normal
26 27 21.16 65.41 Normal
27 28 30.90 70.87 Normal
28 29 39.60 49.05 Anomaly
29 30 24.39 45.44 Normal

Average Temperature : 30.47
Average Humidity : 58.95
Maximum Temperature : 39.6
Minimum Temperature : 20.18
Total Messages : 30
Anomalies Detected : 8
```



RESULT

The real-time IoT stream analytics system successfully generated sensor data, transmitted it through the MQTT broker, received it using a subscriber, performed stream analytics, detected abnormal temperature values, and visualized the results in Google Colab. The experiment demonstrates how MQTT-based IoT systems can be integrated with Python for real-time monitoring and anomaly detection without requiring a local Node-RED installation.